UNIVERSIDAD NACIONAL DE COLOMBIA

Laboratory 4

Quality Attribute Scenarios

Jeisson Andrés Vergara Vargas

Large-Scale Software Architecture

2026-I

---

1. Objective

The objective of this lab is to study six quality attribute scenarios for the ERTMS — two per attribute, each at a different level of abstraction — and to apply the same scenario structure to the architectural model of the team's own course project.

NFP: Non-Functional Property. SoS: System of Systems. MAS: Movement Authority Service.

---

2. Prerequisites

- Completion of Lab 3 (the annotated `model.arch` must be available).

---

3. Background

3.1. Quality Scenario Structure

A quality scenario makes a quality requirement precise and testable by describing a concrete situation in six elements:

Element	Role in the scenario
Source	The actor or condition that generates the stimulus — a user, a subsystem, a hardware fault, time.
Stimulus	The specific event or condition that the system must handle.
Artifact	The part of the system — the SoS, a subsystem, a component, a connector — that receives the stimulus.

Element	Role in the scenario
Environment	The operational state of the system when the stimulus arrives — normal operation, degraded mode, peak load.
Response	The observable behavior the system exhibits in reaction to the stimulus.
Response Measure	The quantitative criterion that determines whether the response is acceptable.

A scenario without a response measure is not a scenario — it is an intention. The measure is what connects an architectural decision to a verifiable outcome.

3.2. Two Levels of Scenario in the ERTMS

The ERTMS is a System of Systems composed of multiple constituent systems — among them the ticketing system, the Movement Authority Service (MAS), and the physical layer of onboard units, sensors, and actuators. Quality requirements exist at both levels:

- **SoS scenario**: the artifact is the entire SoS or one of its constituent systems viewed from the outside. The response measure describes emergent, aggregate behavior — what an operator or regulator observes across the whole.
- **System scenario**: the artifact is a specific component or connector inside one constituent system. The response measure targets a local element and is traceable to an architectural decision made inside that system.

The relationship between the two levels is not symmetric. A SoS-level requirement can be met even when individual systems fail — if the architecture distributes responsibility and isolates faults. Conversely, a system that satisfies all its local scenarios may still violate the SoS requirement if the failure propagation paths between systems are not controlled.

---

4. ERTMS Quality Scenarios

4.1. Availability

QS-AV-SoS-1 — Availability at SoS level

> **Situation.** The Tickets subsystem — one of the constituent systems of the ERTMS SoS — suffers a complete infrastructure failure during normal operations. Train validation and movement authority continue to be needed across the network.

Element	Description
Source	Tickets subsystem (infrastructure fault)
Stimulus	The subsystem becomes fully unavailable — no requests can be processed or responded to
Artifact	ERTMS System of Systems

Element	Description
Environment	Normal operation — all other constituent systems are running; transaction load is within normal parameters
Response	The SoS continues operating; the Movement Authority Service and the physical layer remain functional; ticketing operations are suspended but do not affect train safety or scheduling
Response Measure	The SoS maintains at least **92% of its normal operational capacity**, measured as the ratio of successfully completed non-ticketing transactions during the failure window to the baseline transaction rate

Reading the scenario. The 92% figure reflects the architectural decision to isolate the Tickets subsystem from the safety-critical path. Train movement authority and physical control do not depend on ticketing availability; a Tickets failure therefore affects throughput but not safety. The 8% degradation represents the proportion of total SoS workload attributable to ticketing operations under normal load.

QS-AV-SYS-1 — Availability at System level

> **Situation.** A server hosting the Movement Authority Service (`mas`) — annotated `availability: critical, performance: realtime` in `model.arch` — fails during normal operation. The `mas` is responsible for computing and issuing movement authorities to onboard units; its unavailability directly affects train safety.

Element	Description
Source	Physical server (hardware fault)
Stimulus	The server becomes unavailable — the `mas` process terminates unexpectedly
Artifact	`mas` component (logic tier, `model.arch`)
Environment	Normal operation — trains are active across the network; `mas` is processing movement authority requests at its normal frequency
Response	The system detects the failure; a standby instance of `mas` takes over; in-flight requests are retried; no movement authority is issued for a train whose authority has not been confirmed
Response Measure	The `mas` resumes processing within **500 ms** of failure detection; no confirmed movement authority is lost; the number of trains that receive an incorrect or missing authority is **zero**

Reading the scenario. The `mas` is annotated `availability: critical` — the strictest level in the DSL. The 500 ms recovery threshold derives from the `realtime` performance annotation: a component that must respond in real time cannot tolerate a recovery window measured in seconds. The zero-loss requirement on movement authorities reflects the safety nature of this component — unlike ticketing, there is no acceptable degradation path.

4.2. Performance

QS-PE-SoS-1 — Performance at SoS level

> **Situation.** Rush hour begins simultaneously across all cities served by the ERTMS SoS. Passengers attempt to purchase tickets and validate travel passes at a rate significantly above the daily average. All constituent systems are under load at the same time.

Element	Description
Source	Passenger population across the SoS service area
Stimulus	Concurrent arrival of 2 000 purchase and validation transactions within a 10-second window
Artifact	ERTMS System of Systems
Environment	Peak load — all constituent systems are operational; network conditions are normal
Response	The SoS distributes the load across available instances; no transaction is dropped; queued transactions are processed in order
Response Measure	At least **95% of transactions** complete within **3 000 ms** end-to-end; no more than **1%** of transactions fail due to capacity exhaustion

Reading the scenario. The 3 000 ms threshold maps to the `interactive` performance level defined in the DSL grammar. Purchase and validation transactions are not safety-critical — a passenger waiting three seconds for a ticket confirmation is inconvenient, not dangerous. The 1% failure ceiling acknowledges that a small number of retries is tolerable under peak load, while still holding the architecture accountable for overall throughput.

QS-PE-SYS-1 — Performance at System level

> **Situation.** A formation of trains on a high-frequency corridor begins transmitting position updates at maximum rate. The `position_time_ms` microservice — annotated `availability: critical, performance: realtime` — receives a burst of concurrent update events from multiple `onboard_unit` components via connectors annotated `timeout_ms: 200, encrypted: true`.

Element	Description
Source	Multiple `onboard_unit` components (physical tier)
Stimulus	Burst of 500 concurrent position update events arriving within 1 second
Artifact	`position_time_ms` microservice (logic tier, `model.arch`)
Environment	Normal operation — the microservice is already processing a baseline load of position updates; no prior degradation

Element	Description
Response	Each update is received, processed, and persisted to `position_time_db`; the `mas` is notified of any position that triggers a movement authority recalculation; no update is silently dropped
Response Measure	Every position update is processed within **200 ms** of receipt; no update is lost; the `mas` receives notification within **300 ms** of any position change that requires authority recalculation

Reading the scenario. The 200 ms threshold comes directly from the connector annotation `timeout_ms: 200` on the `onboard_unit → position_time_ms` data flow. It is the tightest timing constraint in the architecture and reflects the physical reality that train position changes rapidly. The 300 ms notification bound for the `mas` adds a small margin for the internal processing chain without compromising safety.

4.3. Security

QS-SE-SoS-1 — Security at SoS level

> **Situation.** An external actor — outside the ERTMS SoS boundary — attempts to inject unauthorized movement authority commands by sending malformed requests through the public-facing portal. The goal is to cause a train to receive an invalid authority.

Element	Description
Source	External attacker (unauthorized third party)
Stimulus	Injection of 1 000 malformed or spoofed movement authority requests through the `portal` component (presentation tier, `security: public`)
Artifact	ERTMS System of Systems — specifically the path from the public boundary to the `mas`
Environment	Normal operation — no prior security incident; the SoS is processing regular traffic concurrently
Response	All injected requests are rejected at the `api_gateway` boundary; none reaches the `mas`; a security event is logged for each rejected request; no legitimate transaction is disrupted
Response Measure	**Zero** injected requests reach the `mas`; the `api_gateway` rejects all requests that do not carry valid authentication; the detection and logging of the attack completes within **1 000 ms** of the first injected request

Reading the scenario. The `portal` is annotated `security: public` — it is the lowest-security boundary in the architecture. The `api_gateway` sits between the public boundary and the logic tier and is annotated `security: restricted`; it is the first enforcement point. The scenario tests whether the security gradient defined in `model.arch` — from `public` at the edge to `restricted` at the safety core — actually blocks unauthorized access rather than merely labeling it.

QS-SE-SYS-1 — Security at System level

> **Situation.** An attacker with access to the internal network attempts a man-in-the-middle attack on the control command channel between the `mas` and the `onboard_unit`. This connector is annotated `encrypted: true, timeout_ms: 200` in `model.arch`.

Element	Description
Source	Internal attacker (insider threat or compromised node)
Stimulus	Interception and attempted modification of a `control_command` message on the `mas → onboard_unit` connector
Artifact	`mas → onboard_unit` connector (control_command type, `model.arch`)
Environment	Normal operation — trains are active; the connector is processing live movement authority commands
Response	The tampered message is detected via integrity verification; the `onboard_unit` discards the message; the `mas` is notified of the failed delivery; the train holds its last confirmed authority until a new authenticated command arrives
Response Measure	**Zero** tampered messages are accepted by any `onboard_unit`; detection of tampering occurs within the **200 ms** timeout window; no train acts on an unverified command

Reading the scenario. The `encrypted: true` annotation on this connector is the architectural decision that this scenario tests. Encryption alone prevents passive eavesdropping; the scenario goes further by testing active tampering — which requires integrity verification, not just confidentiality. The 200 ms bound comes from the connector's `timeout_ms` annotation: a command that cannot be verified within that window must be treated as invalid, not held indefinitely.

---

5. Analysis

The six scenarios form three pairs, one per attribute. In each pair, the SoS-level scenario establishes an aggregate requirement and the system-level scenario traces it to a specific architectural element. The table below summarizes the relationship:

Attribute	SoS scenario	System scenario	Architectural link
Availability	SoS operates at ≥ 92% when Tickets fails	`mas` recovers in ≤ 500 ms with zero authority loss	`mas` is on the critical path for the non-ticketing 92%
Performance	95% of transactions complete in ≤ 3 000 ms	Position updates processed in ≤ 200 ms	`position_time_ms` is on the critical path for validation transactions

Attribute	SoS scenario	System scenario	Architectural link
Security	Zero injected requests reach `mas`	Zero tampered commands accepted by `onboard_unit`	`api_gateway` and the encrypted connector are the two enforcement boundaries

Answer the following questions. Each answer must be two to four paragraphs and must cite specific elements from `model.arch` or from the scenarios above.

Question 1. The SoS availability scenario (QS-AV-SoS-1) specifies a 92% operational capacity when Tickets fails. The system availability scenario (QS-AV-SYS-1) addresses a `mas` failure — a different subsystem entirely. Would a `mas` failure violate the 92% threshold? Justify your answer by examining what the 92% figure covers and what it does not. Then propose the response measure that a SoS-level availability scenario for a `mas` failure would need.

Question 2. The performance pair (QS-PE-SoS-1 and QS-PE-SYS-1) contains two timing thresholds: 3000 ms at the SoS level and 200 ms at the system level. Is the SoS threshold achievable if the system threshold is violated? Trace the transaction path from the `portal` or `onboard_unit` through the architecture to identify which components sit between the two measures and which of their annotations are most relevant.

Question 3. The security pair (QS-SE-SoS-1 and QS-SE-SYS-1) tests two different boundaries: the public edge (`portal` → `api_gateway`) and the internal control channel (`mas` → `onboard_unit`). Identify one component or connector in `model.arch` that is *not* directly referenced by either scenario but whose annotation suggests it should be covered by a third security scenario. Write the artifact, response, and response measure for that scenario.

---

6. Delivery

The delivery consists of a single document:

- **Report** (≤ 2 pages): answers to the three analysis questions (Section 5).

All ERTMS references must cite component or connector names exactly as they appear in `model.arch`.

6.1. Submission Format

The delivery must be submitted via Google Classroom. File in *.pdf* format named **lab4-team-X**, where X = team (1a, 1b, ... 2a, 2b, ...).

6.2. Delivery Deadline

Friday, April 24, 2026, before 23h59.